

**FUNDAÇÃO ESCOLA DE COMÉRCIO ÁLVARES PENTEADO
FECAP**

CENTRO UNIVERSITÁRIO ÁLVARES PENTEADO

ANÁLISE E DESENVOLVIMENTO DE SISTEMAS

**ANNA PAULA ALVES SILVA
DILLY MARTINS DA SILVA
LAURA RAYSSA SOUZA ARAÚJO
MATHEUS GAJEWSKI DE MELO
RAFAELA CARVALHO BARCOS MELLO**

**Segunda Entrega PI Algoritmo e logica de programação
PROFESSOR ORIENTADOR: EDUARDO SAVINO GOMES**

**São Paulo
2026**

Observações

Após o envio da primeira entrega identificamos uma necessidade da aplicação de modularização e vetorização ao código base do aplicativo desktop sendo desenvolvido nesse projeto.

A inclusão de informações em formato de lista(vetores) se mostrou de extrema importância no projeto visto que uma rede de ensino pode ter diversos endereços de IPs conectados.

A separação do código em módulos de procedimentos distintos foi identificado como importante para a rotina de ação do programa, visto que sua atuação não segue um caminho linear, onde a escola pode se cadastrar antes de liberar o acesso aos jogos aos estudantes ou ela pode ter se cadastrado anteriormente e apenas precisa iniciar uma nova sessão.

Vale lembrar que o código enviado agora é um protótipo; decidimos focar no funcionamento principal, pois a parte visual ainda será integrada e mudará a forma como o usuário interage com o sistema.

Por fim, os números de acessos e o limite de IPs usados aqui são apenas exemplos para teste e não refletem os dados reais que serão usados no dia a dia.

Estrutura de Vetorização

A vetorização consiste no armazenamento de diversos dados do mesmo tipo em uma única estrutura de lista. No contexto do projeto, ela é aplicada no processo de adicionar a quantidade de IP's válidos e inserir cada um individualmente.

```
67
68 //Estrutura de vetorização, usada para inserir a quantidade de IP's
69     Console.WriteLine("Quantos IPs deseja autorizar? ");
70     qtdIPs = int.Parse(Console.ReadLine());
71 //Aqui a estrutura de repetição foi usada para a inserção dos IP's
72     for (int i = 0; i < qtdIPs; i++)
73     {
74         Console.WriteLine($"Digite o IP {i + 1}: ");
75         listaIPs.Add(Console.ReadLine());
76     }
```

ESTRUTURA DE VETORIZAÇÃO - ENTRADA E SAÍDA		
TIPO	DADO	DESCRIÇÃO
Entrada	Tamanho da Lista	Comprimento do vetor
Entrada	IP Autorizado	Controle de acesso por rede

Pseudocódigo - Estrutura Sequencial

Algoritmo_Estrutura_de_Vetorizacao

```
escreva("Quantos IPs deseja autorizar (máx 5)? ")
leia(qtdIPs)
```

```
// VETORIZAÇÃO: Armazenando múltiplos IPs
para i de 1 ate qtdIPs faca
    escreva("Digite o IP ", i, ": ")
    leia(listalPs[i])
fimpara
```

Estrutura de Modularização

A modularização é a técnica de dividir um programa complexo em partes menores e independentes, chamadas de módulos. Cada módulo fica responsável por uma tarefa, neste caso, em duas funções, cadastro e login

1. Módulo Cadastro

Utilizado para a escola se cadastrar na plataforma, com informações como, CNPJ, Nome da Escola, Senha e a lista de IP's autorizados.

```

60 // --- MÓDULO 1: CADASTRO
61 //Utilizado para a escola que não possui cadastro, realiza-o
62 static void RealizarCadastro(ref string cnpj, ref string senha,
63     ref string escola, ref string nPacote, ref int limite)
64 {
65     Console.WriteLine("\n--- CADASTRO ADMINISTRATIVO LUMINA
66     LEARNING ---");
67     Console.Write("CNPJ da escola: ");
68     cnpj = Console.ReadLine();
69     Console.Write("Nome da escola: ");
70     escola = Console.ReadLine();
71     Console.Write("Senha: ");
72     senha = Console.ReadLine();
73
74     //Estrutura de vetorização, usada para inserir a quantidade
75     //de IP's
76     Console.Write("Quantos IPs deseja autorizar? ");
77     if (int.TryParse(Console.ReadLine(), out qtdIPs))
78     {
79         //Aqui a estrutura de repetição foi usada para a
80         //inserção dos IP's
81         for (int i = 0; i < qtdIPs; i++)
82         {
83             Console.Write($"Digite o IP {i + 1}: ");
84             listalPs.Add(Console.ReadLine());
85         }
86     }
87
88     // validação de Pacote com loop
89     while (nPacote == "INVALIDO")
90     {
91         Console.WriteLine("Escolha o pacote: (1)Bronze (2)Prata
92         (3)Ouro (4)Diamante");
93         string opcao = Console.ReadLine();
94
95         switch (opcao)
96         {
97             case "1":
98                 nPacote = "Bronze";
99                 limite = 100;
100                 break;
101             case "2":
102                 nPacote = "Prata";
103                 limite = 300;
104                 break;
105             case "3":
106                 nPacote = "Ouro";
107                 limite = 400;
108                 break;
109             case "4":
110                 nPacote = "Diamante";
111
112         }
113     }
114
115     Console.WriteLine($"Escola cadastrada com sucesso no pacote
116     {nPacote}!");
117 }
```

MÓDULO CADASTRO - ENTRADA E SAÍDA		
TIPO	DADO	DESCRIÇÃO
Entrada	CNPJ	Identificação da escola
Entrada	Nome da Escola	Nome da instituição
Entrada	Senha	Acesso ao sistema
Entrada	Tamanho da Lista	Comprimento do vetor
Entrada	IP Autorizado	Controle de acesso por rede
Entrada	Opção do pacote	Escolha do plano
Saída	Nome do pacote	Nome do plano escolhido
Saída	Limite de acessos	Quantidade de acessos permitidos por mês

Pseudocódigo - Módulo Cadastro

```
// --- MÓDULO 1: CADASTRO ---
// Função: Registra a escola, seus IPs autorizados e define o limite do pacote.
procedimento realizar_cadastro(var cnpj, senha, escola, nPacote: caractere; var
limite: inteiro)
var
    opcao, i: inteiro
inicio
    escreval("--- CADASTRO ADMINISTRATIVO LUMINA LEARNING ---")
    escreva("CNPJ da escola: ")
    leia(cnpj)
    escreva("Nome da escola: ")
    leia(escola)
    escreva("Senha: ")
    leia(senha)

    escreva("Quantos IPs deseja autorizar (máx 5)? ")
    leia(qtdIPs)

    // VETORIZAÇÃO: Armazenando múltiplos IPs
    para i de 1 ate qtdIPs faca
        escreva("Digite o IP ", i, ": ")
        leia(listas[i])
    fimpara
```

```

// DEFINIÇÃO DO PACOTE COM VALIDAÇÃO
enquanto nPacote == Invalido
    escreva("Escolha o pacote: (1)Bronze (2)Prata (3)Ouro (4)Diamante")
    leia(opcao)

    escolha opcao
        caso 1
            nPacote <- "Bronze"
            limite <- 100
        caso 2
            nPacote <- "Prata"
            limite <- 300
        caso 3
            nPacote <- "Ouro"
            limite <- 400
        caso 4
            nPacote <- "Diamante"
            limite <- 1000
        outrocaso
            nPacote <- "INVALIDO"
            escreva("Erro: Opção inválida! Você precisa escolher uma das opções
listadas (1 a 4).")
        fimescolha
    fimenquanto
    escreva("Escola cadastrada com sucesso no pacote ", nPacote, "!")
fimprocedimento

```

2. Módulo Login

Usado para iniciar uma sessão com os dados previamente informados no cadastro.

```

116
117 // --- MÓDULO 2: LOGIN ---
118 static bool RealizarLogin(string c_cad, string s_cad)
119 {
120     for (int tentativas = 1; tentativas <= 3; tentativas++)
121     {
122         Console.WriteLine($"\\n--- LOGIN (Tentativa {tentativas}/3) ---");
123         Console.Write("CNPJ: ");
124         string c_log = Console.ReadLine();
125         Console.Write("Senha: ");
126         string s_log = Console.ReadLine();
127         Console.Write("IP Local: ");
128         string ip_log = Console.ReadLine();
129
130         // Verificação de IP no Vetor (Lista)
131         bool ipValido = listaIPs.Contains(ip_log);
132
133         if (c_log == c_cad && s_log == s_cad && ipValido)
134         {
135             Console.WriteLine("Acesso liberado!");
136             return true;
137         }
138         else
139         {
140             Console.WriteLine("Dados inválidos ou IP não autorizado!");
141         }
142     }
143     return false;
144 }
145 }

```

MÓDULO LOGIN - ENTRADA E SAÍDA		
TIPO	DADO	DESCRIÇÃO
Entrada	CNPJ	Identificação da escola
Entrada	Senha	Acesso ao sistema
Entrada	IP Autorizado	Controle de acesso por rede

Pseudocódigo - Módulo Cadastro

// --- MÓDULO 2: LOGIN ---

// Função: Valida as credenciais e verifica se o IP atual está no vetor de IPs cadastrados.

funcao realizar_login(c_cad, s_cad: caractere): logico

var

 c_log, s_log, ip_log: caractere

 i, tentativas: inteiro

 ipValido: logico

inicio

 tentativas <- 1

 enquanto (tentativas <= 3) faca

 escreval("--- LOGIN (Tentativa ", tentativas, "/3) ---")

 escreva("CNPJ: "); leia(c_log)

 escreva("Senha: "); leia(s_log)

 escreva("IP Local: "); leia(ip_log)

 ipValido <- falso

 // VETORIZAÇÃO: Percorrendo o vetor para validar o IP

 para i de 1 ate qtdIPs faca

 se (listaIPs[i] = ip_log) entao

 ipValido <- verdadeiro

 fimse

 fimpara

 se (c_log = c_cad) e (s_log = s_cad) e (ipValido) entao

 escreval("Acesso liberado!")

 retorne verdadeiro

 senao

 escreval("Dados inválidos ou IP não autorizado!")

 tentativas <- tentativas + 1

 fimse

fimenquanto

retorne falso
fimfuncao